IUT de Paris - Rives de Seine
Université Paris Cité

Rapport de projet - SAÉ S5.A.01

IQ15Y010 SAÉ Développement avancé

ECO STOCK

Année : 2025-2026

Groupe : 304.1

Réalisé par :

- NGUYEN Phuong
- LIMONTA Evan
- ATCHEKZAI Ismaïl
- BOUZIDI Rayen

Encadrant :

- M. Ouziri Mourad : Référent Technique

Date de rendu : Vendredi 20 février 2026

IUT de Paris - Rives de Seine
Université Paris Cité

Table des matières

1. Introduction 2
2. Présentation de l'existant (version S5) 4
2.1 Gestion des utilisateurs 5
2.2 Côté commerçant 5
2.3 Côté Consommateur 6
2.4 Recettes et accompagnement à la consommation 6
2.5 Dimension associative 6
3. Objectifs de la S6 7
3.1 Ajouter une évolution fonctionnelle pertinente 7
3.2 Améliorer les performances techniques 8
3.3 Renforcer la sécurité de l'application 8
3.4 Structurer le travail d'équipe et la documentation 9
4. Maintenance évolutive fonctionnelle : système d'avis et de notation 10
4.1 Objectifs de la fonctionnalité 11
4.2 Fonctionnement côté utilisateur 11
4.3 Règles métier et mécanismes anti-abus 12
4.4 Implémentation backend 12
4.5 Impact utilisateur 14
5. Évolution technologique et amélioration des performances 14
5.1 Utilisation de Prisma ORM 14
5.1.1 Approche brownfield 16
5.1.2 Migrations clés des requêtes 16
5.2 Mise en place d'un cache Redis 16
5.2.1 Données cachées et TTL 16
5.2.2 Impact mesuré sur les performances 17
6. Maintenance de sécurité : audits, vulnérabilités et corrections 17
6.1 Analyse statique avec SonarQube : avant et après 18
6.1.1 Corrections de fiabilité (Reliability C → A) 18
6.1.2 Corrections des Security Hotspots (E → A) 19
6.2 Analyse dynamique avec OWASP ZAP 19

6.3 Bénéfices de la démarche sécurité 20
7. Tests et validation du projet 20
8. Module IA – Scanner d'ingrédients 21
8.1 Architecture globale 21
8.1.1 Pourquoi une architecture multi-couches ? 22
8.1.2 Entraînement du CNN (train_cnn.py) 22
8.1.3 Détection Out-of-Distribution (OOD) 22
8.1.4 Limitation documentée 22
9. Résultats et limites 23
9.1 Résultats obtenus 23
9.2 Limites actuelles 24
10. Synthèse + conclusion 25
10.1 Synthèse 25
10.2 Conclusion générale 26

1. Introduction

Le gaspillage alimentaire représente aujourd'hui un enjeu majeur, à la fois écologique, économique et social. Chaque année, plusieurs millions de tonnes de produits encore consommables sont jetées, notamment au niveau des commerces de proximité. Face à cette problématique, le projet EcoStock a été conçu comme une solution numérique visant à faciliter la revente des invendus alimentaires, tout en accompagnant les utilisateurs dans leur consommation afin de limiter le gaspillage à domicile.

EcoStock est une application mobile qui met en relation des commerçants locaux et des consommateurs. Elle permet aux vendeurs de publier leurs produits invendus avec leurs informations essentielles (prix, description, date limite de consommation, horaires de retrait), tandis que les utilisateurs peuvent rechercher des produits proches de leur position, effectuer leurs achats directement via l'application et bénéficier de suggestions de recettes adaptées aux produits récupérés. Le projet intègre également une dimension solidaire grâce à des comptes associations donnant accès à des offres spécifiques.

Lors de la SAÉ S5, une première version fonctionnelle de l'application a été développée. Celle-ci incluait déjà les principales fonctionnalités métier : gestion des utilisateurs, interface commerçant, interface consommateur, système de recettes intelligentes, recherche géolocalisée et intégration d'associations. Une architecture complète avait été mise en place, reposant sur une application mobile React Native, un backend Node.js/Express et une base de données PostgreSQL avec l'intégration de services externes tels que Stripe pour le paiement, Nominatim pour la géolocalisation et Cloudinary pour la gestion des images.

La SAÉ S6 s'inscrit dans une logique de maintenance évolutive et corrective. L'objectif n'était plus uniquement de développer de nouvelles fonctionnalités mais également d'améliorer la qualité globale du projet existant. Cette phase a porté sur quatre points principaux :

- L'ajout d'une nouvelle fonctionnalité métier majeure (système d'avis et de notation),
- L'amélioration des performances via l'optimisation des accès à la base de données et la mise en cache,
- Le renforcement de la sécurité de l'application à l'aide d'outils d'audit et de bonnes pratiques,

- La stabilisation du code à travers la correction de vulnérabilités et d’anomalies détectées.

Cette évolution a permis de transformer EcoStock en une application plus robuste plus sécurisée et plus proche des besoins réels des utilisateurs, tout en respectant les contraintes techniques et les exigences du cahier des charges.

Dans ce rapport, nous présenterons d’abord l’état initial du projet puis les évolutions apportées durant la SAÉ S6, tant sur le plan fonctionnel que technique. Nous détaillerons ensuite les améliorations de performance et de sécurité mises en œuvre ainsi l’utilisation de l'IA avant de conclure sur les résultats obtenus et la conclusion.

2. Présentation de l’existant (version S5)

Avant d’aborder les évolutions réalisées durant la SAÉ S6, il est important de rappeler l’état initial du projet tel qu’il existait à la fin de la SAÉ S5. Cette première version constituait une base fonctionnelle complète, autour de laquelle ont été construites les améliorations ultérieures.

L’application EcoStock reposait sur une architecture client–serveur composée de trois éléments principaux :

- Une application mobile développée en React Native, destinée aux consommateurs, commerçants et associations,
- Un backend développé en Node.js avec Express, exposant une API REST,
- Une base de données PostgreSQL assurant la persistance des données.

Plusieurs services externes étaient également intégrés afin d’enrichir les fonctionnalités :

- Stripe pour la gestion des paiements,
- Cloudinary pour l’hébergement et l’optimisation des images produits,
- Nominatim (OpenStreetMap) pour la géolocalisation des commerces,
- AsyncStorage pour la persistance locale du panier utilisateur.

2.1 Gestion des utilisateurs

Le système de gestion des utilisateurs permettait la création de différents types de comptes :

- Consommateurs,
- Commerçants,
- Associations.

Chaque utilisateur disposait d’un profil personnel et d’un système d’authentification sécurisé. Selon son rôle, l’interface et les fonctionnalités accessibles variaient.

Les commerçants pouvaient gérer leur boutique, publier leurs produits invendus et suivre leurs ventes, tandis que les consommateurs pouvaient consulter les produits disponibles, passer commande et accéder à leur historique d'achats. Les associations bénéficiaient d'un accès spécifique à certaines offres, renforçant l'aspect solidaire du projet.

2.2 Côté commerçant

L'espace vendeur permettait aux commerçants de :

- Ajouter de nouveaux produits avec photo, description, prix initial et prix réduit,
- Préciser la date limite de consommation (DLC) et les horaires de retrait,
- Gérer leurs stocks en temps réel,
- Consulter leurs ventes et l'état des commandes (en attente, confirmée, récupérée).

Cette interface visait à simplifier la publication des invendus et à offrir une vision claire de l'activité du commerce.

2.3 Côté Consommateur

Du côté utilisateur, l'application proposait :

- Une recherche intelligente multicritères (prix, catégorie, DLC),
- Un tri par proximité grâce à une formule de calcul de distance,
- Un panier persistant,
- Un paiement sécurisé via Stripe,
- Un historique des commandes.

L'utilisateur pouvait visualiser les produits disponibles autour de lui, consulter les détails d'un produit, effectuer son achat directement depuis l'application et se rendre ensuite chez le commerçant pour récupérer sa commande.

2.4 Recettes et accompagnement à la consommation

Une fonctionnalité importante du projet consistait à accompagner l'utilisateur après l'achat. À partir des produits récupérés, l'application proposait des recettes adaptées ainsi que des suggestions d'ingrédients disponibles à proximité.

Cette approche visait à aller au-delà de la simple revente d'invendus, en favorisant une consommation réelle et responsable des produits, afin de limiter le gaspillage alimentaire à domicile.

2.5 Dimension associative

EcoStock intégrait également une dimension sociale. Les associations disposaient de comptes spécifiques leur permettant d'accéder à certaines offres, contribuant ainsi à redistribuer les invendus à des publics en situation de précarité.

Cette fonctionnalité renforçait l'impact solidaire du projet, en complément de l'aspect économique et environnemental.

À l'issue de la SAÉ S5, EcoStock constituait donc une application complète et fonctionnelle. Toutefois, plusieurs axes d'amélioration ont été identifiés, notamment en termes d'expérience utilisateur, de confiance envers les commerçants, de performances et de sécurité.

Ces constats ont servi de point de départ à la SAÉ S6, dont l'objectif principal était d'enrichir le projet existant par une maintenance évolutive et corrective.

3. Objectifs de la S6

La SAÉ S6 s'inscrit dans une logique de **maintenance évolutive et corrective** d'un projet existant. Contrairement à la SAÉ S5, dont l'objectif principal était le développement initial d'une application fonctionnelle, la SAÉ S6 visait à améliorer, renforcer et professionnaliser une solution déjà opérationnelle.

L’objectif global n’était donc pas simplement d’ajouter une fonctionnalité supplémentaire, mais de faire évoluer EcoStock vers une version plus mature, plus performante et plus sécurisée, en respectant les exigences d’un projet informatique réel.

Les objectifs de cette phase peuvent être regroupés en quatre points principaux.

3.1 Ajouter une évolution fonctionnelle pertinente

La première exigence de la SAÉ S6 consistait à proposer une maintenance évolutive fonctionnelle, c'est-à-dire une amélioration concrète du produit existant.

Après analyse de l'application développée en S5, un manque important a été identifié : l'absence d'un mécanisme de confiance entre utilisateurs et commerçants. Or, dans toute application de type marketplace, la transparence et la réputation jouent un rôle central dans la décision d'achat.

Nous avons donc choisi d'implémenter un **système d'avis et de notation des commerces**, permettant :

- Aux utilisateurs de noter un commerce sur une échelle de 1 à 5 étoiles,
- D'ajouter un commentaire optionnel,
- De consulter la moyenne des avis avant achat.

Cette évolution répond à un besoin réel et améliore significativement l'expérience utilisateur.

3.2 Améliorer les performances techniques

Un second objectif majeur était d'améliorer la qualité technique et les performances de l'application.

Bien que la version S5 soit fonctionnelle, certaines requêtes pouvaient être optimisées, notamment au niveau :

- De l'accès à la base de données,
- Du traitement des requêtes fréquemment appelées,
- De la gestion des données répétitives.

Pour répondre à cet objectif, nous avons :

- Optimisé l'accès aux données via l'utilisation de Prisma ORM,
- Intégré un système de cache Redis,
- Réalisé des mesures comparatives avant/après afin de quantifier les gains de performance.

Cette approche nous a permis d'améliorer les temps de réponse et de rendre l'application plus réactive.

3.3 Renforcer la sécurité de l'application

Un autre axe central de la SAÉ S6 était la sécurisation du projet.

Dans un contexte professionnel, la sécurité d'une application ne peut être négligée, notamment lorsqu'elle manipule :

- Des données personnelles,
- Des informations de paiement,
- Des accès utilisateurs authentifiés.

Nous avons donc mis en place une démarche complète comprenant :

- Une analyse statique du code via SonarQube,
- Une analyse dynamique via OWASP ZAP,
- La correction des vulnérabilités identifiées,
- L'application de bonnes pratiques de secure coding en référence aux standards OWASP.

Cette étape a permis d'améliorer la fiabilité globale du projet et de réduire les risques de vulnérabilités exploitables.

3.4 Structurer le travail d'équipe et la documentation

Enfin, la SAÉ S6 imposait également une organisation rigoureuse du travail d'équipe.

Nous avons structuré le projet autour :

- D'une répartition claire des tâches (fonctionnel, performance, sécurité),
- D'un suivi via Git,
- De validations croisées,
- D'une documentation technique détaillée,
- Et d'une préparation à la soutenance orale.

Cette démarche visait à reproduire un contexte de développement professionnel, où la qualité du code, la traçabilité des modifications et la capacité à justifier ses choix sont essentielles.

Ainsi, la SAÉ S6 ne constitue pas seulement une extension du projet S5, mais une véritable montée en maturité d'EcoStock, tant sur le plan fonctionnel que technique et organisationnel.

4. Maintenance évolutive fonctionnelle : système d'avis et de notation

Dans le cadre de la SAÉ S6, une maintenance évolutive fonctionnelle a été mise en œuvre afin d'améliorer l'expérience utilisateur et la fiabilité de la plateforme EcoStock. Après analyse de la version développée en S5, un manque important a été identifié : l'absence de mécanisme permettant aux utilisateurs d'évaluer les commerces.

Or, dans une application de type marketplace, la confiance joue un rôle central dans la prise de décision. Sans indicateur de qualité, l'utilisateur ne dispose d'aucun retour sur le sérieux d'un commerce, le respect des horaires ou la conformité des produits proposés.

Pour répondre à cette problématique, nous avons choisi d'implémenter un système d'avis et de notation des commerces, permettant aux consommateurs de partager leur expérience.

4.1 Objectifs de la fonctionnalité

Cette évolution vise plusieurs objectifs :

- Renforcer la transparence entre consommateurs et commerçants ;
- Permettre aux utilisateurs de comparer les commerces avant achat ;
- Encourager les vendeurs à maintenir un service de qualité ;
- Créer une boucle d'amélioration continue basée sur les retours clients.

Cette fonctionnalité apporte ainsi une réelle valeur ajoutée au projet, en transformant EcoStock d'une simple plateforme d'achat d'invendus en un écosystème participatif.

4.2 Fonctionnement côté utilisateur

Depuis la fiche d'un produit ou d'un commerce, l'utilisateur peut consulter les avis existants et accéder à une vue détaillée affichant :

- La note moyenne du commerce,
- Le nombre total d'avis,
- Les commentaires laissés par les autres utilisateurs.

Un acheteur peut également laisser son propre avis en attribuant une note de 1 à 5 étoiles et en ajoutant un commentaire optionnel.
La note moyenne est recalculée automatiquement après chaque ajout ou modification.

Chaque utilisateur dispose aussi d'un écran "Mes avis", lui permettant de consulter ou supprimer ses propres évaluations.

Du côté commerçant, une vue dédiée permet de consulter l'ensemble des avis reçus sur sa boutique.

4.3 Règles métier et mécanismes anti-abus

Afin de garantir la fiabilité du système et d'éviter les abus, plusieurs règles ont été mises en place côté backend et base de données :

- Un seul avis par utilisateur et par commerce, assuré par une contrainte UNIQUE (user_id, commerce_id) ;
- Mise à jour automatique de l'avis existant en cas de nouvel envoi (upsert Prisma) ;
- Impossibilité pour un vendeur de noter sa propre boutique, vérifiée côté serveur ;
- Validation stricte des notes entre 1 et 5 (contrôle backend + contrainte CHECK PostgreSQL) ;
- Suppression d'un avis autorisée uniquement par son auteur.

Chaque avis est lié à un compte utilisateur, avec date de création visible, assurant une traçabilité complète.

Ces mécanismes empêchent le spam, les doublons et l'auto-notation, garantissant un système d'évaluation crédible.

4.4 Implémentation backend

Endpoint	Rôle
POST /api/reviews	Créer ou mettre à jour un avis (upsert)
GET /api/reviews/commerce/:id	Récupérer les avis d'un commerce + stats calculées
GET /api/reviews/my-reviews	Consulter ses propres avis en tant qu'acheteur

GET /api/reviews/shop-reviews	Consulter les avis reçus sur sa boutique (vendeur uniquement)
DELETE /api/reviews/:reviewId	Supprimer son propre avis
GET /api/reviews/check/:commerceId	Vérifier si l'utilisateur a déjà noté ce commerce

Le calcul de la note moyenne est effectué via Prisma grâce aux fonctions d'agrégation (_avg, _count). Les statistiques associées à chaque commerce sont ensuite mises en cache à l'aide de Redis avec un TTL de 5 minutes et invalidées automatiquement après chaque modification.

Cette approche permet de limiter les requêtes répétitives vers PostgreSQL tout en garantissant des données à jour.

4.5 Impact utilisateur

L'ajout du système d'avis améliore significativement l'expérience globale :

- Les utilisateurs peuvent choisir un commerce en toute confiance ;
- Les mauvaises surprises sont réduites grâce aux retours visibles ;
- Les commerçants sont incités à respecter leurs engagements ;
- Les retours clients permettent d'identifier rapidement les points d'amélioration.

Cette fonctionnalité renforce ainsi la crédibilité de la plateforme et favorise une utilisation durable du service.

5. Évolution technologique et amélioration des performances

En parallèle de l'évolution fonctionnelle, la SAÉ S6 a été l'occasion de renforcer l'architecture technique du projet EcoStock afin d'améliorer ses performances, sa maintenabilité et sa robustesse.

Bien que l'application développée en S5 soit pleinement fonctionnelle, certaines opérations présentaient des marges d'optimisation, notamment au niveau des accès à la base de données et du traitement de données fréquemment sollicitées. L'objectif était donc de réduire les temps de réponse tout en rendant le code plus lisible et plus facilement maintenable.

Deux évolutions majeures ont été mises en place : l'utilisation de Prisma ORM et l'intégration d'un système de cache Redis.

5.1 Utilisation de Prisma ORM

Le backend utilisait initialement le module pg (node-postgres) avec des requêtes SQL brutes. Ce fonctionnement présentait plusieurs inconvénients : pas de typage ni d'autocomplétion, gestion manuelle des transactions, code difficile à refactoriser. La migration vers Prisma ORM répond à ces problèmes tout en préservant la possibilité d'exécuter du SQL brut pour les requêtes complexes.

Avant – pg / SQL brut	Après – Prisma ORM
Requêtes SQL écrites à la main	Client type-safe avec autocomplétion
Aucun typage ni autocomplétion	Transactions natives $transaction()
Transactions manuelles pool.connect()	upsert, findMany, aggregate intégrés
Difficile à refactoriser	Schéma auto-généré par introspection
—	$queryRaw disponible pour SQL complexe

5.1.1 Approche brownfield

Plutôt que de réécrire le schéma manuellement, l'introspection Prisma a été utilisée (npx prisma db pull) pour générer le schéma à partir de la base PostgreSQL existante. Cette approche conserve intactes toutes les fonctions PostgreSQL : triggers, contraintes CHECK, génération des numéros de commande. Le schéma final comprend 15 modèles.

5.1.2 Migrations clés des requêtes

- **findMany() remplace SELECT brut :** prisma.categories.findMany({ select: { id: true, nom: true }, orderBy: { nom: 'asc' } }) remplace le db.query('SELECT id, nom FROM categories ORDER BY nom') initial.

- **$transaction() remplace BEGIN/COMMIT/ROLLBACK :** La gestion manuelle avec pool.connect(), BEGIN, ROLLBACK dans le catch et client.release() est remplacée par une fonction callback atomique. Si une opération échoue, Prisma rollback automatiquement.

- **upsert() pour les avis :** Le pattern check-then-insert (deux requêtes + logique conditionnelle) est remplacé par prisma.reviews.upsert() en une seule opération atomique, en réutilisant la contrainte UNIQUE [user_id, commerce_id].

Les requêtes complexes utilisant des fonctionnalités PostgreSQL avancées (calcul Haversine, scoring CASE WHEN, ARRAY_AGG, GREATEST) sont conservées en SQL brut via $queryRaw. Un point technique notable : les fonctions d'agrégation PostgreSQL retournent des BigInt que JSON.stringify() ne sait pas sérialiser. La correction BigInt.prototype.toJSON = function() { return Number(this); } est ajoutée dans le singleton PrismaClient.

5.2 Mise en place d'un cache Redis

Redis a été intégré comme couche de cache optionnelle. Si Redis est indisponible, l'application continue de fonctionner normalement en lisant directement PostgreSQL. Toutes les fonctions de cache échouent silencieusement. Le serveur Node démarre même si Redis est absent grâce à lazyConnect: true, avec une retry strategy de 3 tentatives (backoff 200 ms, 400 ms, 2 000 ms).

5.2.1 Données cachées et TTL

Données	Clé Redis	TTL	Invalidation
Catégories produits	categories:all	1 heure	Jamais (données statiques)

Recettes paginées	recipes:page:{limit}:{offset}	10 min	—
Recette individuelle	recipe:{id}	30 min	—
Recherche d'ingrédients	ingredients:search:{query}	15 min	—
Stats avis commerce	commerce:stats:{commerceId}	5 min	Après création/suppression d'avis

---

5.2.2 Impact mesuré sur les performances

Des tests réalisés avec Postman sur la route des avis d'un commerce montrent l'impact concret du cache : 789 ms pour la première exécution (lecture PostgreSQL), 36 ms pour la seconde (lecture cache Redis), soit une réduction de 95% du temps de réponse. À titre de comparaison, la même route atteignait 900 ms avec l'ancien code SQL brut avant la migration Prisma — qui l'a elle-même réduit à 789 ms. Le gain cumulé Prisma + Redis est donc significatif.

6. Maintenance de sécurité : audits, vulnérabilités et corrections

La sécurisation de l'application EcoStock constitue un axe central de la SAÉ S6. Dans un contexte réel, une application manipulant des données utilisateurs et des transactions financières doit impérativement respecter des standards de sécurité élevés.

L'objectif de cette phase était d'identifier les failles potentielles, d'améliorer la fiabilité du code et de mettre en place des bonnes pratiques de secure coding.

Pour cela, une démarche structurée a été adoptée, reposant sur trois étapes principales :

- Analyse statique du code avec SonarQube ;
- Analyse dynamique de l'application avec OWASP ZAP ;
- Correction des vulnérabilités identifiées et amélioration continue du code.

6.1 Analyse statique avec SonarQube : avant et après

Avant nos corrections, SonarQube relevait des problèmes significatifs sur le code. Après traitement, toutes les métriques sont passées au niveau A.

Métrique	Avant corrections	Après corrections
Security	A (0 issues)	A (0 issues)
Reliability	C (30 open issues)	A (0 open issues)
Maintainability	A (77 issues)	A (69 issues)
Security Hotspots	E (3 hotspots critiques)	A (0 hotspot)
Duplications	6.0% sur 21k lignes	6.0% sur 21k lignes

6.1.1 Corrections de fiabilité (Reliability C → A)

La note C en Reliability regroupait 30 problèmes identifiés, de natures variées. Nous présentons ici deux exemples représentatifs des corrections effectuées :

Boolean leaked values (S6840) : En JSX React Native, un rendu conditionnel {value && <Component />} peut afficher '0' ou 'NaN' à l'écran si value est falsy mais pas strictement

false. Correction : ajout d'une double négation (!!maVariable) dans le JSX pour forcer une évaluation booléenne stricte. Exemple : {product.category_name && ...} devient {!!product.category_name && ...}.

Promise void return (S6544) : Des fonctions async utilisées comme callbacks onPress risquaient de planter silencieusement. Une Promise non gérée dans un événement bouton ne lève pas d'erreur visible mais peut laisser l'état de l'application dans un état incohérent. Correction : encapsulation via l'opérateur void — () => void (async () => { await ... })() — pour isoler la Promise et l'exécuter correctement.

6.1.2 Corrections des Security Hotspots (E → A)

Les 3 hotspots critiques ont été traités. Voici les 5 vulnérabilités corrigées, référencées selon le Top 10 OWASP 2021 :

1. A02 – Cryptographic Failures (Vulnérabilité de Transport) : L'API du serveur IA autorisait des origines en clair (http://) écrites en dur dans le code, exposant les données à des interceptions de type Man-in-the-Middle. Correction : suppression des protocoles non sécurisés codés en dur, la politique CORS est désormais pilotée strictement par la variable d'environnement du serveur.

2. A04 – Insecure Design (Vulnérabilité ReDoS – Déni de Service) : La regex validant les emails pouvait saturer le processeur par backtracking exponentiel si une chaîne piégée était envoyée (ReDoS). Correction : réécriture stricte de la regex en isolant les domaines ([^\s@.]+) pour empêcher les chevauchements.

3. A05 – Security Misconfiguration (CORS trop permissif) : L'API et le serveur IA acceptaient les requêtes de n'importe quel site web extérieur (Access-Control-Allow-Origin: *). Cela expose l'API à des attaques CSRF cross-origin. Correction : implémentation d'une liste blanche filtrant les requêtes, n'autorisant que nos URL frontend et les tunnels ngrok de développement.

4. A05 – Security Misconfiguration (Fuite d'empreinte technique) : Le serveur dévoilait sa technologie sous-jacente via le header X-Powered-By: Express sur chaque réponse HTTP. Un attaquant peut exploiter cette info pour cibler des failles connues d'Express/Node.js. Correction : désactivation via app.disable('x-powered-by').

5. A07 – Identification and Authentication Failures (Credentials hard-codés) : Des secrets JWT étaient écrits en clair dans les fichiers de tests (process.env.JWT_SECRET = 'test-secret-key'). Un accès au code source suffisait pour connaître le secret de signature des tokens. Correction : remplacement par une référence à la variable d'environnement avec fallback CI uniquement.

6.2 Analyse dynamique avec OWASP ZAP

Un audit dynamique (pentest) a été réalisé avec OWASP ZAP sur le backend en cours d'exécution. L'outil lance des requêtes automatisées — injections, fuzzing, scans de configuration — pour éprouver la résistance de l'API. Résultat : aucune faille critique (rouge) ni moyenne (orange) n'a été détectée. Une seule alerte mineure a été remontée : l'absence du header de sécurité X-Content-Type-Options, catégorisée OWASP_2021_A05 Security Misconfiguration. Sans cet en-tête, des navigateurs anciens peuvent interpréter le type MIME d'une réponse autrement que déclaré (MIME-sniffing), potentiellement ouvrant la porte à des injections de contenu. Correction : ajout d'un middleware Express qui force la valeur nosniff sur toutes les réponses, validé en interrogeant directement le serveur.

6.3 Bénéfices de la démarche sécurité

Cette maintenance de sécurité a apporté plusieurs bénéfices :

- Réduction drastique des risques de vulnérabilités exploitables ;
- Amélioration de la qualité globale du code ;
- Meilleure conformité aux standards OWASP ;
- Prise de conscience collective de l’importance du secure coding.

Cette étape a également permis à l’équipe de se familiariser avec des outils professionnels largement utilisés dans l’industrie.

7. Tests et validation du projet

Cette maintenance de sécurité a apporté plusieurs bénéfices : une réduction drastique des risques de vulnérabilités exploitables, une amélioration de la qualité globale du code, une meilleure conformité aux standards OWASP, ainsi qu’une prise de conscience collective de l’importance du *secure coding*. Cette étape a également permis à l’équipe de se familiariser avec des outils professionnels largement utilisés dans l’industrie.

Dans la continuité de cette démarche de robustesse, et afin de garantir la fiabilité des nouvelles fonctionnalités introduites lors de la SAÉ S6 ainsi que la stabilité globale de l’application, une stratégie de tests automatisés a été mise en place. Ces tests permettent de vérifier le bon fonctionnement des composants principaux, de prévenir

les régressions et d'assurer la cohérence des données après les différentes évolutions apportées.

L'approche adoptée repose sur des tests unitaires et fonctionnels, principalement réalisés avec le framework Jest et son extension *ts-jest* pour l'exécution de tests TypeScript.

8. Module IA – Scanner d'ingrédients

Dans le cadre de la SAÉ S6, un module d'intelligence artificielle a été intégré afin d'enrichir l'expérience utilisateur et de faciliter l'identification des produits alimentaires.

Ce module permet à l'utilisateur de scanner un aliment à l'aide de la caméra du téléphone, puis d'obtenir une reconnaissance automatique de l'ingrédient. Cette fonctionnalité vise à simplifier la recherche de produits ou de recettes, notamment pour les utilisateurs rencontrant des difficultés à identifier certains aliments.

8.1 Architecture globale

Le scanner est implémenté comme un microservice Python Flask indépendant (port 5001). Le backend Node.js joue le rôle de proxy entre l'application mobile et ce serveur IA, ce qui permet de faire tourner les modèles de deep learning en Python sans impacter le serveur principal.

Couche	Technologie	Rôle	Seuil de confiance
1 – YOLO v8	ultralytics	Détection d'objets : fruits, légumes, viandes. Retourne bounding boxes + score. Support GPU auto (CUDA → CPU).	0.4
2 – CNN ResNet-18	PyTorch	Classification des épices et condiments. Transfer learning + augmentation d'images. Détection OOD par distance aux prototypes.	0.5
3 – EasyOCR	easyocr	Lecture de texte sur emballages. Comparaison à un dictionnaire de 80+ ingrédients en FR, EN, IT, ES (9 catégories alimentaires).	0.3

8.1.1 Pourquoi une architecture multi-couches ?

Chaque approche prise isolément a des limites. YOLO détecte bien les aliments bruts à forme distinctive mais pas les épices en poudre. ResNet-18 spécialisé sur les épices compense ce point : léger, rapide à entraîner, suffisant pour les ~15 classes d'épices ciblées. L'OCR intervient en dernier recours en lisant les étiquettes des emballages. La fusion des trois sources produit une liste d'ingrédients avec un score de confiance composite.

8.1.2 Entraînement du CNN (train_cnn.py)

Le pipeline d'entraînement comprend 5 étapes : collecte d'images par classe → augmentation (rotation, zoom, flip, luminosité) → entraînement ResNet-18 avec Adam + CrossEntropy → calcul des prototypes de classe (centroïdes dans l'espace des features) → calcul des rayons OOD. Le transfer learning est utilisé : les poids pré-entraînés sur ImageNet sont conservés et seule la dernière couche fully-connected est remplacée.

8.1.3 Détection Out-of-Distribution (OOD)

Sans mécanisme OOD, le CNN forcerait une prédiction dans ses classes connues même sur une image hors-sujet. Chaque classe possède un prototype (centroïde de ses vecteurs de features) et un rayon de décision (distance maximale observée en entraînement). Si la distance entre l'image et le prototype dépasse le rayon multiplié par une marge, l'image est rejetée comme hors-distribution. Le moteur de fusion agrège ensuite les résultats avec une règle de scoring : YOLO > 0.9 confirme directement, YOLO 0.5–0.9 déclenche une vérification croisée OCR, et chaque source supplémentaire qui confirme un ingrédient lui ajoute un bonus de +5%.

8.1.4 Limitation documentée

Conformément au cahier des charges, le système ne reconnaît que les aliments bruts et non transformés. Les aliments visuellement proches (épices en poudre de couleur similaire) peuvent ne pas être distingués de façon fiable.

9. Résultats et limites

9.1 Résultats obtenus

À l'issue de la SAÉ S6, EcoStock a évolué vers une application plus complète, plus performante et plus sécurisée.

Sur le plan fonctionnel, l'intégration du système d'avis et de notation permet désormais aux utilisateurs de partager leur expérience, de consulter les évaluations des commerces et de faire des choix plus éclairés avant achat. Cette évolution renforce significativement la transparence de la plateforme et améliore la confiance entre consommateurs et commerçants.

Sur le plan technique, l'adoption de Prisma ORM et l'intégration du cache Redis ont permis de réduire les temps de réponse et d'optimiser l'accès aux données. Les mesures réalisées montrent une nette amélioration des performances, avec des requêtes passant d'environ 900 ms à moins de 800 ms, puis à quelques dizaines de millisecondes lors des appels suivants grâce au mécanisme de cache.

La maintenance de sécurité a également permis d'augmenter fortement la qualité du projet. Les audits SonarQube et OWASP ZAP ont conduit à la correction de nombreuses vulnérabilités et incohérences, faisant passer les indicateurs de fiabilité et de sécurité

au niveau A. L'application respecte désormais davantage les standards OWASP et adopte des pratiques de secure coding.

Enfin, l'intégration du module IA de reconnaissance d'ingrédients apporte une dimension innovante au projet. Bien que perfectible, ce module démontre la capacité de l'application à évoluer vers des fonctionnalités intelligentes facilitant l'usage au quotidien.

Dans l'ensemble, les objectifs fixés en début de SAÉ S6 ont été atteints : EcoStock est aujourd'hui une application plus robuste, plus fiable et plus proche d'un produit professionnel.

9.2 Limites actuelles

Malgré ces améliorations, certaines limites subsistent :

- Le modèle IA repose sur un jeu de données encore restreint, ce qui impacte parfois la précision des prédictions ;
- Le système d'avis pourrait être enrichi par une modération automatique ou manuelle ;
- Les recommandations restent principalement basées sur la proximité et les produits disponibles, sans exploitation poussée de l'historique utilisateur ;
- L'application n'intègre pas encore de notifications push pour alerter des nouvelles offres ou rappels de commandes.

Ces points constituent des axes d'amélioration identifiés pour de futures versions.

10. Synthèse + conclusion

10.1 Synthèse

Axe	Prévu au CDC	Réalisé	Statut
Système d'avis	Notes 1-5, commentaires, note moyenne, consultation	6 endpoints, règles anti-abus, cache Redis intégré	✓ Complet
Sécurité SonarQube	≥ 5 vulnérabilités corrigées	5 failles OWASP corrigées, Reliability C → A, Security Hotspots E → A	✓ Complet
Pentest OWASP ZAP	Tests d'intrusion dynamiques	Aucune faille critique, 1 alerte mineure corrigée (MIME-sniffing)	✓ Complet
Module IA	Reconnaissance d'ingrédients par photo	Pipeline YOLO + CNN + OCR avec fusion et détection OOD	✓ Complet
Prisma ORM	Réduction erreurs SQL, meilleure lisibilité	Migration brownfield, 15 modèles, approche hybride $queryRaw	✓ Complet

Cache Redis	Optimisation données fréquentes	Dégradation gracieuse, -95% latence sur route avis	✓ Complet
SSO OAuth2 / OIDC	Optionnel	Non réalisé dans cette itération	— Non réalisé

Les quatre points principaux du cahier des charges S6 ont été entièrement réalisés. La seule évolution non implémentée est le SSO OAuth2/OIDC, présenté comme optionnel dans le cahier des charges.

10.2 Conclusion générale

La SAÉ S6 a permis de faire évoluer EcoStock d'une application fonctionnelle vers un produit plus mature, intégrant à la fois des améliorations métier, techniques et sécuritaires.

L'ajout du système d'avis et de notation a renforcé la transparence et la confiance des utilisateurs, tandis que les optimisations apportées via Prisma et Redis ont significativement amélioré les performances. La démarche de sécurisation, basée sur des outils professionnels tels que SonarQube et OWASP ZAP, a permis de corriger de nombreuses vulnérabilités et d'adopter des pratiques conformes aux standards actuels.

L'intégration du module IA ouvre quant à elle la voie à des usages plus intelligents et personnalisés, illustrant le potentiel d'évolution du projet.

Au-delà des aspects techniques, ce projet a permis de travailler sur un cycle de développement complet, incluant conception, évolution, sécurisation et validation, dans un cadre proche des pratiques professionnelles. EcoStock représente ainsi une expérience enrichissante, tant sur le plan technique que méthodologique, et constitue une base solide pour de futures améliorations.